

Give an $O(VE)$ -time algorithm for computing the transitive closure of a directed graph $G = (V, E)$. Assume that $|V| = O(E)$ and that the graph is represented with adjacency lists.

23.2-9

Suppose that it takes $f(|V|, |E|)$ time to compute the transitive closure of a directed acyclic graph, where f is a monotonically increasing function of both $|V|$ and $|E|$. Show that the time to compute the transitive closure $G^* = (V, E^*)$ of a general directed graph $G = (V, E)$ is then $f(|V|, |E|) + O(V + E^*)$.

23.3 Johnson's algorithm for sparse graphs

Johnson's algorithm finds shortest paths between all pairs in $O(V^2 \lg V + VE)$ time. For sparse graphs, it is asymptotically faster than either repeated squaring of matrices or the Floyd-Warshall algorithm. The algorithm either returns a matrix of shortest-path weights for all pairs of vertices or reports that the input graph contains a negative-weight cycle. Johnson's algorithm uses as subroutines both Dijkstra's algorithm and the Bellman-Ford algorithm, which Chapter 22 describes.

Johnson's algorithm uses the technique of *reweighting*, which works as follows. If all edge weights w in a graph $G = (V, E)$ are nonnegative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex. With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(V^2 \lg V + VE)$. If G has negative-weight edges but no negative-weight cycles, first compute a new set of nonnegative edge weights so that Dijkstra's algorithm applies. The new set of edge weights $\hat{w}$ must satisfy two important properties:

1. For all pairs of vertices $u, v \in V$, a path p is a shortest path from u to v using weight function w if and only if p is also a shortest path from u to v using weight function $\hat{w}$.
2. For all edges (u, v) , the new weight $\hat{w}(u, v)$ is nonnegative.

As we'll see in a moment, preprocessing G to determine the new weight function $\hat{w}$ takes $O(VE)$ time.

Preserving shortest paths by reweighting

The following lemma shows how to reweight the edges to satisfy the first property above. We use δ to denote shortest-path weights derived from weight function w and $\hat{\delta}$ to denote shortest-path weights derived from weight function $\hat{w}$.

Lemma 23.1 (Reweighting does not change shortest paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $h : V \rightarrow \mathbb{R}$ be any function mapping vertices to real numbers. For each edge $(u, v) \in E$, define

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) . \quad (23.10)$$

Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be any path from vertex v_0 to vertex v_k . Then p is a shortest path from v_0 to v_k with weight function w if and only if it is a shortest path with weight function $\hat{w}$. That is, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$. Furthermore, G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$.

Proof We start by showing that

$$\hat{w}(p) = w(p) + h(v_0) - h(v_k) . \quad (23.11)$$

We have

$$\begin{aligned} \hat{w}(p) &= \sum_{i=1}^k \hat{w}(v_{i-1}, v_i) \\ &= \sum_{i=1}^k (w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)) \\ &= \sum_{i=1}^k w(v_{i-1}, v_i) + h(v_0) - h(v_k) \quad (\text{because the sum telescopes}) \\ &= w(p) + h(v_0) - h(v_k) . \end{aligned}$$

Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$. Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$. Thus, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.

Finally, we show that G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$. Consider any cycle $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$. By equation (23.11),

$$\begin{aligned}\hat{w}(c) &= w(c) + h(v_0) + h(v_k) \\ &= w(c),\end{aligned}$$

and thus c has negative weight using w if and only if it has negative weight using $\hat{w}$. ■

Producing nonnegative weights by reweighting

Our next goal is to ensure that the second property holds: $\hat{w}(u, v)$ must be nonnegative for all edges $(u, v) \in E$. Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, we'll see how to make a new graph $G' = (V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$. To incorporate the new vertex s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$. Since no edges enter s , no shortest paths in G' , other than those with source s , contain s . Moreover, G' has no negative-weight cycles if and only if G has no negative-weight cycles. Figure 23.6(a) shows the graph G' corresponding to the graph G of Figure 23.1.

Now suppose that G and G' have no negative-weight cycles. Define the function $h(v) = \delta(s, v)$ for all $v \in V'$. By the triangle inequality (Lemma 22.10 on page 633), we have $h(v) \leq h(u) + w(u, v)$ for all edges $(u, v) \in E'$. Thus, by defining reweighted edge weights $\hat{w}$ according to equation (23.10), we have $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$, thereby satisfying the second property. Figure 23.6(b) shows the graph G' from Figure 23.6(a) with reweighted edges.

Computing all-pairs shortest paths

Johnson's algorithm to compute all-pairs shortest paths uses the Bellman-Ford algorithm (Section 22.1) and Dijkstra's algorithm (Section 22.3) as subroutines. The pseudocode appears in the procedure JOHNSON on page 666. It assumes implicitly that the edges are stored in adjacency lists. The algorithm returns the usual $|V| \times |V|$ matrix $D = (d_{ij})$, where $d_{ij} = \delta(i, j)$, or it reports that the input graph contains a negative-weight cycle. As is typical for an all-pairs shortest-paths algorithm, it assumes that the vertices are numbered from 1 to $|V|$.

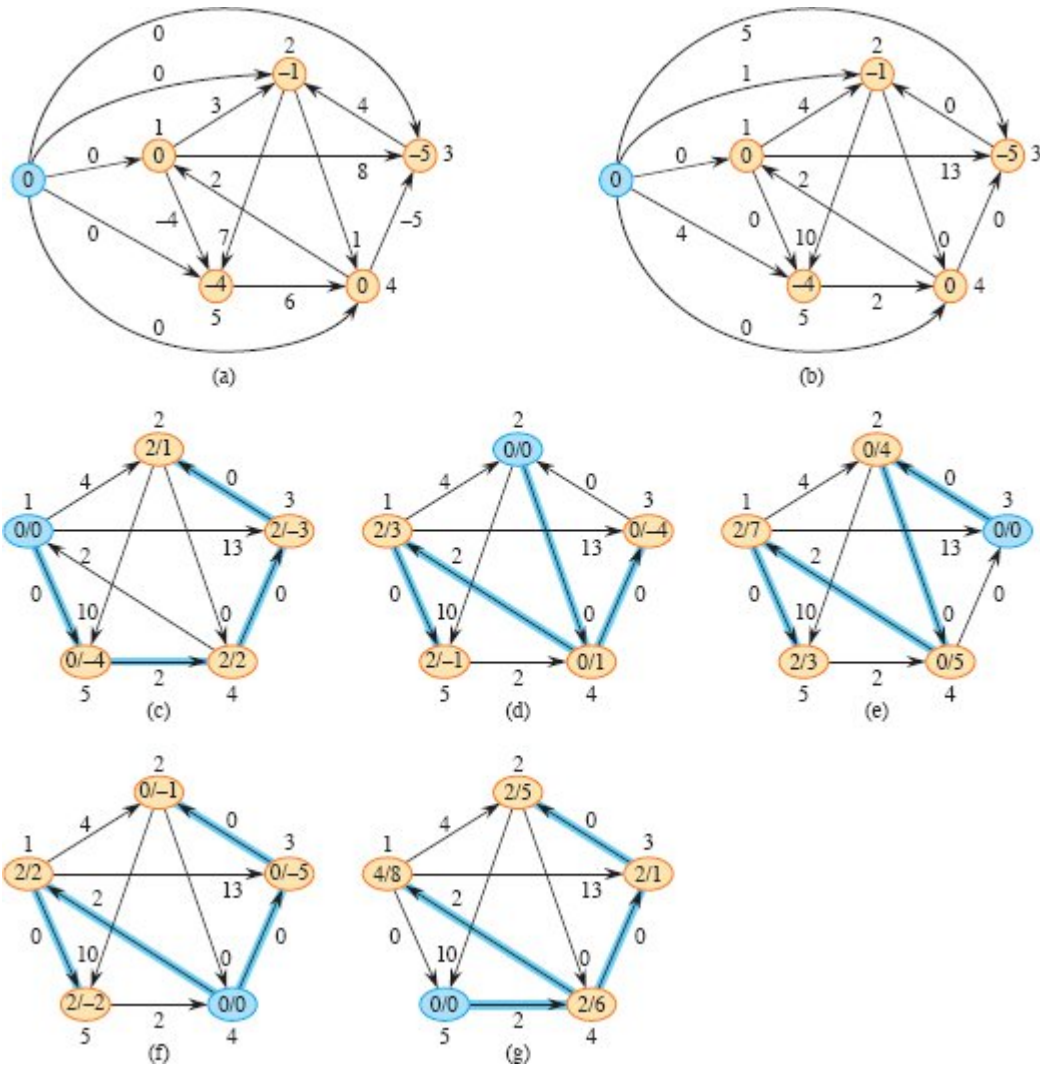


Figure 23.6 Johnson's all-pairs shortest-paths algorithm run on the graph of Figure 23.1. Vertex numbers appear outside the vertices. **(a)** The graph G' with the original weight function w . The new vertex s is blue. Within each vertex v is $h(v) = \delta(s, v)$. **(b)** After reweighting each edge (u, v) with weight function $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$. **(c)–(g)** The result of running Dijkstra's algorithm on each vertex of G using weight function $\hat{w}$. In each part, the source vertex u is blue, and blue edges belong to the shortest-paths tree computed by the algorithm. Within each vertex v are the values $\hat{\delta}(u, v)$ and $\delta(u, v)$, separated by a slash. The value $d_{uv} = \delta(u, v)$ is equal to $\hat{\delta}(u, v) + h(v) - h(u)$.

JOHNSON(G, w)

1 compute G' , where $G'.V = G.V \cup \{s\}$,
 $G'.E = G.E \cup \{(s, v) : v \in G.V\}$, and

```

1  $w(s, v) = 0$  for all  $v \in G.V$ 
2 if BELLMAN-FORD( $G', w, s$ ) == FALSE
3   print “the input graph contains a negative-weight cycle”
4 else for each vertex  $v \in G'.V$ 
5     set  $h(v)$  to the value of  $\delta(s, v)$ 
        computed by the Bellman-Ford algorithm
6   for each edge  $(u, v) \in G'.E$ 
7      $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ 
8   let  $D = (d_{uv})$  be a new  $n \times n$  matrix
9   for each vertex  $u \in G.V$ 
10     run DIJKSTRA( $G, \hat{w}, u$ ) to compute  $\hat{\delta}(u, v)$  for all  $v \in G.V$ 
11     for each vertex  $v \in G.V$ 
12        $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$ 
13 return  $D$ 

```

The JOHNSON procedure simply performs the actions specified earlier. Line 1 produces G' . Line 2 runs the Bellman-Ford algorithm on G' with weight function w and source vertex s . If G' , and hence G , contains a negative-weight cycle, line 3 reports the problem. Lines 4–12 assume that G' contains no negative-weight cycles. Lines 4–5 set $h(v)$ to the shortest-path weight $\delta(s, v)$ computed by the Bellman-Ford algorithm for all $v \in V'$. Lines 6–7 compute the new weights $\hat{w}$. For each pair of vertices $u, v \in V$, the **for** loop of lines 9–12 computes the shortest-path weight $\hat{\delta}(u, v)$ by calling Dijkstra’s algorithm once from each vertex in V . Line 12 stores in matrix entry d_{uv} the correct shortest-path weight $\delta(u, v)$, calculated using equation (23.11). Finally, line 13 returns the completed D matrix. Figure 23.6 depicts the execution of Johnson’s algorithm.

If the min-priority queue in Dijkstra’s algorithm is implemented by a Fibonacci heap, Johnson’s algorithm runs in $O(V^2 \lg V + VE)$ time. The simpler binary min-heap implementation yields a running time of $O(VE \lg V)$, which is still asymptotically faster than the Floyd-Warshall algorithm if the graph is sparse.

Exercises

23.3-1

Use Johnson's algorithm to find the shortest paths between all pairs of vertices in the graph of Figure 23.2. Show the values of h and $\hat{w}$ computed by the algorithm.

23.3-2

What is the purpose of adding the new vertex s to V , yielding V' ?

23.3-3

Suppose that $w(u, v) \geq 0$ for all edges $(u, v) \in E$. What is the relationship between the weight functions w and $\hat{w}$?

23.3-4

Professor Greenstreet claims that there is a simpler way to reweight edges than the method used in Johnson's algorithm. Letting $w^* = \min \{w(u, v) : (u, v) \in E\}$, just define $\hat{w}(u, v) = w(u, v) - w^*$ for all edges $(u, v) \in E$. What is wrong with the professor's method of reweighting?

23.3-5

Show that if G contains a 0-weight cycle c , then $\hat{w}(u, v) = 0$ for every edge (u, v) in c .

23.3-6

Professor Michener claims that there is no need to create a new source vertex in line 1 of JOHNSON. He suggests using $G' = G$ instead and letting s be any vertex. Give an example of a weighted, directed graph G for which incorporating the professor's idea into JOHNSON causes incorrect answers. Assume that $\infty - \infty$ is undefined, and in particular, it is not 0. Then show that if G is strongly connected (every vertex is reachable from every other vertex), the results returned by JOHNSON with the professor's modification are correct.

Problems